

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 14 LECTURE 1

MEMORY

MANAGEMENT

SOC 3010

OPERATING SYSTEM

ZONE ALLOCATOR

The Zone Allocator

- The zone allocator is the frontend of the kernel page frame allocator.
- This component must locate a memory zone that includes a number of free page frames large enough to satisfy the memory request.
- This task is not as simple as it could appear at a first glance, because the zone allocator must satisfy several goals:
 - It should protect the pool of reserved page frames
 - It should trigger the page frame reclaiming algorithm when memory is scarce and blocking the current process is allowed; once some page frames have been freed, the zone allocator will retry the allocation.
 - It should preserve the small, precious ZONE_DMA memory zone, if possible.
 - For instance, the zone allocator should be somewhat reluctant to assign page frames in the ZONE_DMA memory zone if the request was for ZONE_NORMAL or ZONE_HIGHMEM page frames.

The Zone Allocator

❑ Allocating a group of page frames

- In Zoned Page Frame Allocator, every request for a group of contiguous page frames is eventually handled by executing the **alloc_pages macro**.
- This macro, in turn, ends up invoking the **_ _alloc_pages() function**, which is the core of the zone allocator.
- It receives three parameters:
 - gfp_mask** The flags specified in the memory allocation request
 - order** The logarithmic size of the group of contiguous page frames to be allocated
 - Zonelist** Pointer to a zonelist data structure describing, in order of preference, the memory zones suitable for the memory allocation

❑ Releasing a group of page frames

- The zone allocator also takes care of releasing page frames; thankfully, releasing memory is a lot easier than allocating it.
- All kernel macros and functions that release page frames rely on the **_free_pages() function**.
- It receives as its parameters the address of the page descriptor of the first page frame to be released (page), and the logarithmic size of the group of contiguous page frames to be released (order).

Memory Area Management

- Consider memory areas with sequences of memory cells having contiguous physical addresses and an arbitrary length.
- The buddy system algorithm adopts the page frame as the basic memory area.
- This is fine for dealing with relatively large memory requests, but how are we going to deal with requests for small memory areas, say a few tens or hundreds of bytes?
- Clearly, it would be quite wasteful to allocate a full page frame to store a few bytes.
- A better approach instead consists of introducing new data structures that describe how small memory areas are allocated within the same page frame.
- A new problem called **internal fragmentation** is caused by a mismatch between the size of the memory request and the size of the memory area allocated to satisfy the request.

Memory Area Management

- **A classical solution to internal fragmentation** (adopted by early Linux versions) consists of providing memory areas whose sizes are geometrically distributed; in other words, the size depends on a power of 2 rather than on the size of the data to be stored.
- In this way, no matter what the memory request size is, we can ensure that the internal fragmentation is always smaller than 50 percent.
- **Following this approach, the kernel creates 13 geometrically distributed lists of free memory areas whose sizes range from 32 to 131, 072 bytes.**
- The buddy system is invoked both to obtain additional page frames needed to store new memory areas and, conversely, to release page frames that no longer contain memory areas.
- A dynamic list is used to keep track of the free memory areas contained in each page frame.

The Slab Allocator

- Running a memory area allocation algorithm on top of the buddy algorithm is not particularly efficient.
- A better algorithm is derived from the slab allocator schema that was adopted for the first time in the Sun Microsystems Solaris 2.4 operating system.
- It is based on the following premises:
 - The type of data to be stored may affect how memory areas are allocated; for instance, when allocating a page frame to a User Mode process, the kernel invokes the `get_zeroed_page( )` function, which fills the page with zeros.
 - The concept of a slab allocator expands upon this idea and views the memory areas as objects consisting of both a set of data structures and a couple of functions or methods called the constructor and destructor. The former initializes the memory area while the latter deinitializes it.

The Slab Allocator

- To avoid initializing objects repeatedly, the slab allocator does not discard the objects that have been allocated and then released but instead saves them in memory.
- When a new object is then requested, it can be taken from memory without having to be reinitialized.
- The kernel functions tend to request memory areas of the same type repeatedly.
- For instance, whenever the kernel creates a new process, it allocates memory areas for some fixed size tables such as the process descriptor, the open file object, and so on.
- When a process terminates, the memory areas used to contain these tables can be reused.
- **Because processes are created and destroyed quite frequently, without the slab allocator, the kernel wastes time allocating and deallocating the page frames containing the same memory areas repeatedly; the slab allocator allows them to be saved in a cache and reused quickly.**

The Slab Allocator

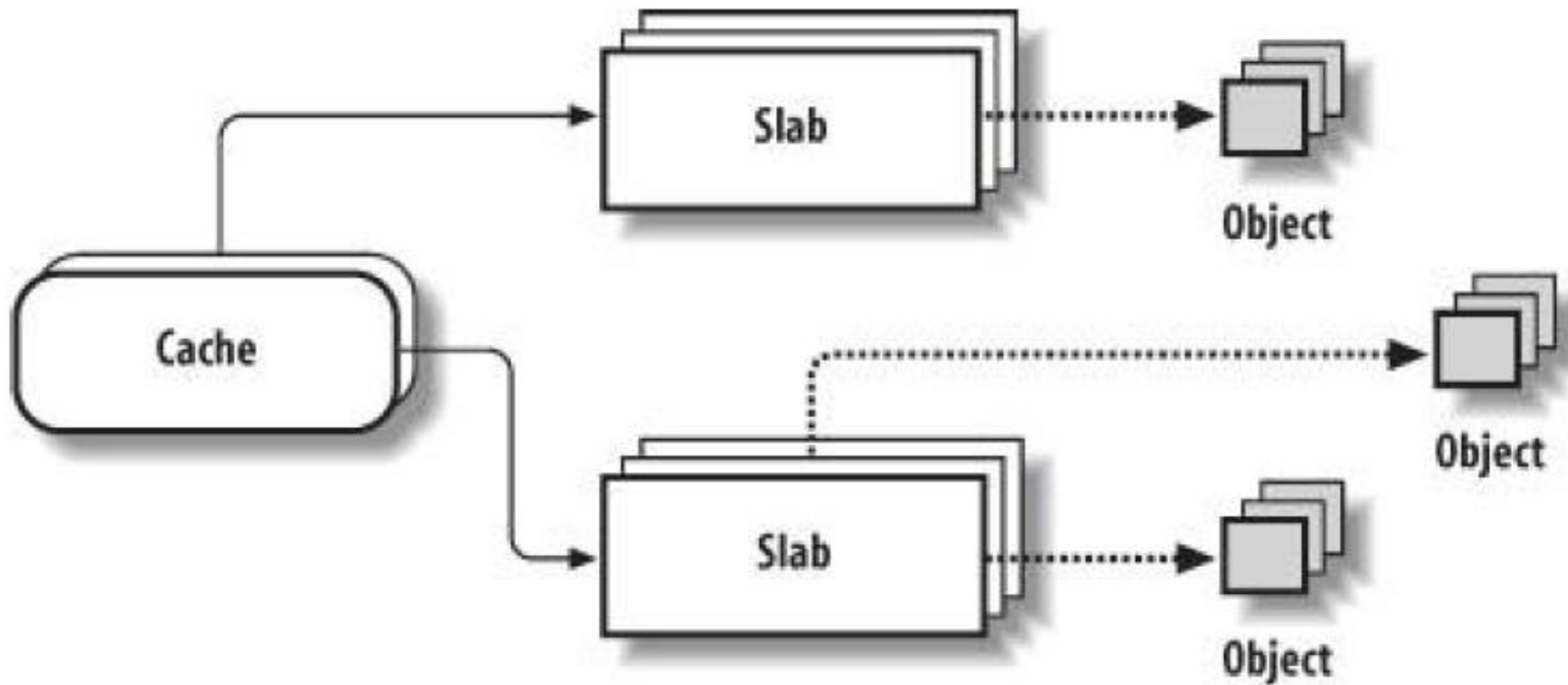
- Requests for memory areas can be classified according to their frequency.
- Requests of a particular size that are expected to occur frequently can be handled most efficiently by creating a set of special-purpose objects that have the right size, thus avoiding internal fragmentation.
- Meanwhile, sizes that are rarely encountered can be handled through an allocation scheme based on objects in a series of geometrically distributed sizes (such as the power-of-2 sizes used in early Linux versions), even if this approach leads to internal fragmentation.

The Slab Allocator

- Hardware cache performance creates an additional reason for limiting calls to the buddy system allocator as much as possible.
- Every call to a buddy system function "dirties" the hardware cache, thus increasing the average memory access time.
- The impact of a kernel function on the hardware cache is called the function footprint; it is defined as the percentage of cache overwritten by the function when it terminates.
- Clearly, large footprints lead to a slower execution of the code executed right after the kernel function, because the hardware cache is by now filled with useless information.
- The slab allocator groups objects into caches .
- Each cache is a "store" of objects of the same type.
- For instance, when a file is opened, the memory area needed to store the corresponding "open file" object is taken from a slab allocator cache named filp (for "file pointer").

The Slab Allocator

- The area of main memory that contains a cache is divided into **slabs**; each slab consists of one or more contiguous page frames that contain both allocated and free objects.
- Figure - The slab allocator components



The kernel periodically scans the caches and releases the page frames corresponding to empty slabs.

The Slab Allocator

❑ SLAB Descriptor

- Each slab of a cache has its own descriptor of type slab illustrated in Table.

The fields of the slab descriptor are:

<u>Type</u>	<u>Name</u>	<u>Description</u>
struct list_head	list	Pointers for one of the three doubly linked list of slab descriptors (either the slabs_full, slabs_partial, or slabs_free list in the kmem_list3 structure of the cache descriptor)
unsigned long	colouroff	Offset of the first object in the slab
void *	s_mem	Address of first object (either allocated or free) in the slab
unsigned int	inuse	Number of objects in the slab that are currently used (not free)
unsigned int	free	Index of next free object in the slab, or BUFCTL_END if there are no free objects left

Slab descriptors can be stored in two possible places:

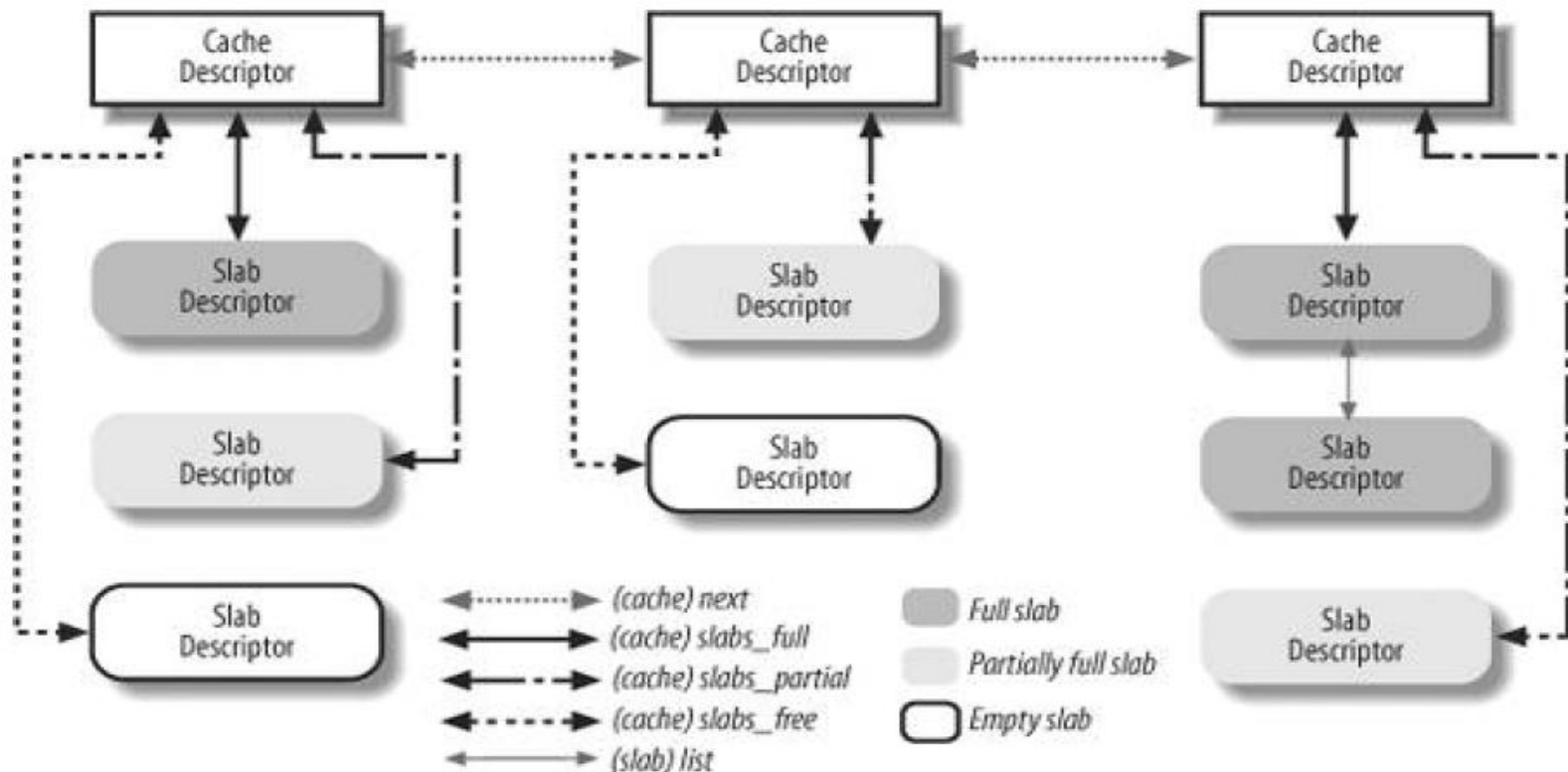
- **External slab descriptor** Stored outside the slab, in one of the general caches not suitable for ISA DMA pointed to by cache_sizes
- **Internal slab descriptor** Stored inside the slab, at the beginning of the first page frame assigned to the slab.

The Slab Allocator

❑ SLAB Descriptor

- The slab allocator chooses the second solution when the size of the objects is smaller than 512MB or when internal fragmentation leaves enough space for the slab descriptor and the object descriptors inside the slab.
- The `CFLGS_OFF_SLAB` flag in the flags field of the cache descriptor is set to one if the slab descriptor is stored outside the slab; it is set to zero otherwise.
- Figure illustrates the major relationships between cache and slab descriptors. Full slabs, partially full slabs, and free slabs are linked in different lists.

Relationship between cache and slab descriptors



The Slab Allocator



□ General and Specific Caches

- Caches are divided into two types: **general and specific**.
- General caches are used only by the slab allocator for its own purposes, while specific caches are used by the remaining parts of the kernel.
- The general caches are:
 - A first cache called **kmem_cache** whose objects are the cache descriptors of the remaining caches used by the kernel.
 - The **cache_cache** variable contains the descriptor of this special cache.
 - Several additional caches contain general purpose memory areas.
 - The range of the memory area sizes typically includes 13 geometrically distributed sizes.
 - A table called **malloc_sizes** (whose elements are of type **cache_sizes**) points to 26 cache descriptors associated with memory areas of size 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384, 32,768, 65,536, and 131,072 bytes. For each size, there are two caches: one suitable for ISA DMA allocations and the other for normal allocations.
 - The **kmem_cache_init()** function is invoked during system initialization to set up the general caches.

The Slab Allocator

- ★ **Specific caches** are created by the **kmem_cache_create()** function.
 - Depending on the parameters, the function first determines the best way to handle the new cache (for instance, whether to include the slab descriptor inside or outside of the slab).
 - It then allocates a cache descriptor for the new cache from the **cache_cache** general cache and inserts the descriptor in the **cache_chain** list of cache descriptors (the insertion is done after having acquired the **cache_chain_sem** semaphore that protects the list from concurrent accesses).
 - It is also possible to destroy a cache and remove it from the **cache_chain** list by invoking **kmem_cache_destroy()**.
 - This function is mostly useful to modules that create their own caches when loaded and destroy them when unloaded.
 - To avoid wasting memory space, the kernel must destroy all slabs before destroying the cache itself.
 - The **kmem_cache_shrink()** function destroys all the slabs in a cache by invoking **slab_destroy()** iteratively
 - The names of all general and specific caches can be obtained at runtime by reading **/proc/slabinfo**; this file also specifies the number of free objects and the number of allocated objects in each cache.

The Slab Allocator

- ❑ **Interfacing the Slab Allocator with the Zoned Page Frame Allocator**
 - When the slab allocator creates a new slab, it relies on the zoned page frame allocator to obtain a group of free contiguous page frames.
 - For this purpose, it invokes the **kmem_getpages()** function, using two parameters having the following meaning:

Cachep	Points to the cache descriptor of the cache that needs additional page frames
Flags	Specifies how the page frame is requested. This set of flags is combined with the specific cache allocation flags stored in the gfpflags field of the cache descriptor.
 - The size of the memory allocation request is specified by the gfporder field of the cache descriptor, which encodes the size of a slab in the cache.

The Slab Allocator

- ❑ **Interfacing the Slab Allocator with the Zoned Page Frame Allocator**
- In the reverse operation, page frames assigned to a slab can be released by invoking the **kmem_freepages() function**.
- The function releases the page frames, starting from the one having the linear address `addr`, that had been allocated to the slab of the cache identified by `cachep`.
- If the current process is performing memory reclaiming (`current->reclaim_state` field not NULL), the `reclaimed_slab` field of the `reclaim_state` structure is properly increased, so that the pages just freed can be accounted for by the page frame reclaiming algorithm.
- Moreover, if the `SLAB_RECLAIM_ACCOUNT` flag is set, the `slab_reclaim_pages` variable is properly decreased.

The Slab Allocator



❑ Allocating a Slab to a Cache

- A newly created cache does not contain a slab and therefore does not contain any free objects.
- New slabs are assigned to a cache only when both of the following are true:
 - A request has been issued to allocate a new object.
 - The cache does not include a free object.
- Under these circumstances, the slab allocator assigns a new slab to the cache by invoking `cache_grow( )`.
- This function calls `kmem_getpages( )` to obtain from the zoned page frame allocator the group of page frames needed to store a single slab; it then calls `alloc_slabmgmt( )` to get a new slab descriptor.
- If the `CFLGS_OFF_SLAB` flag of the cache descriptor is set, the slab descriptor is allocated from the general cache pointed to by the `slabp_cache` field of the cache descriptor; otherwise, the slab descriptor is allocated in the first page frame of the slab.

The Slab Allocator

❑ Releasing a Slab from a Cache

- Slabs can be destroyed in two cases:
 - There are too many free objects in the slab cache
 - A timer function invoked periodically determines that there are fully unused slabs that can be released
- In both cases, the **slab_destroy()** function is invoked to destroy a slab and release the corresponding page frames to the zoned page frame allocator

The Slab Allocator

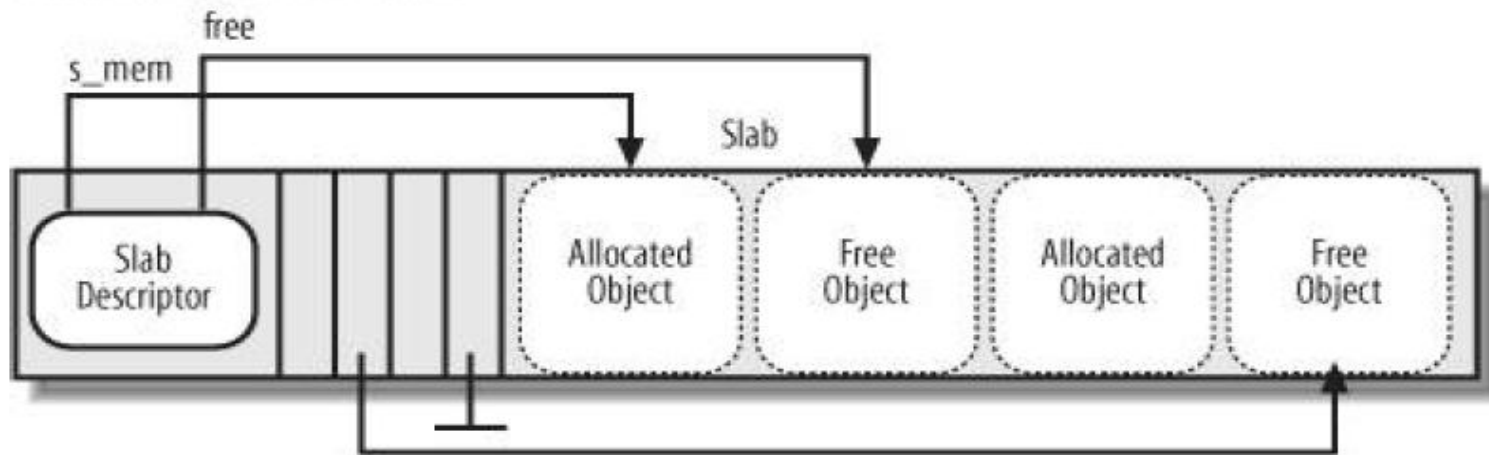
❑ Object Descriptor

- Each object has a short descriptor of type `kmem_bufctl_t`.
- Object descriptors are stored in an array placed right after the corresponding slab descriptor.
- Thus, like the slab descriptors themselves, the object descriptors of a slab can be stored in two possible ways that are illustrated in Figure.
- External object descriptors Stored outside the slab, in the general cache pointed to by the `slabp_cache` field of the cache descriptor.
- The size of the memory area, and thus the particular general cache used to store object descriptors, depends on the number of objects stored in the slab (`num` field of the cache descriptor).
- Internal object descriptors stored inside the slab, right before the objects they describe.
- The first object descriptor in the array describes the first object in the slab, and so on.
- An object descriptor is simply an unsigned short integer, which is meaningful only when the object is free.
- It contains the index of the next free object in the slab, thus implementing a simple list of free objects inside the slab.
- The object descriptor of the last element in the free object list is marked by the conventional value `BUFCTL_END` (`0xffff`).

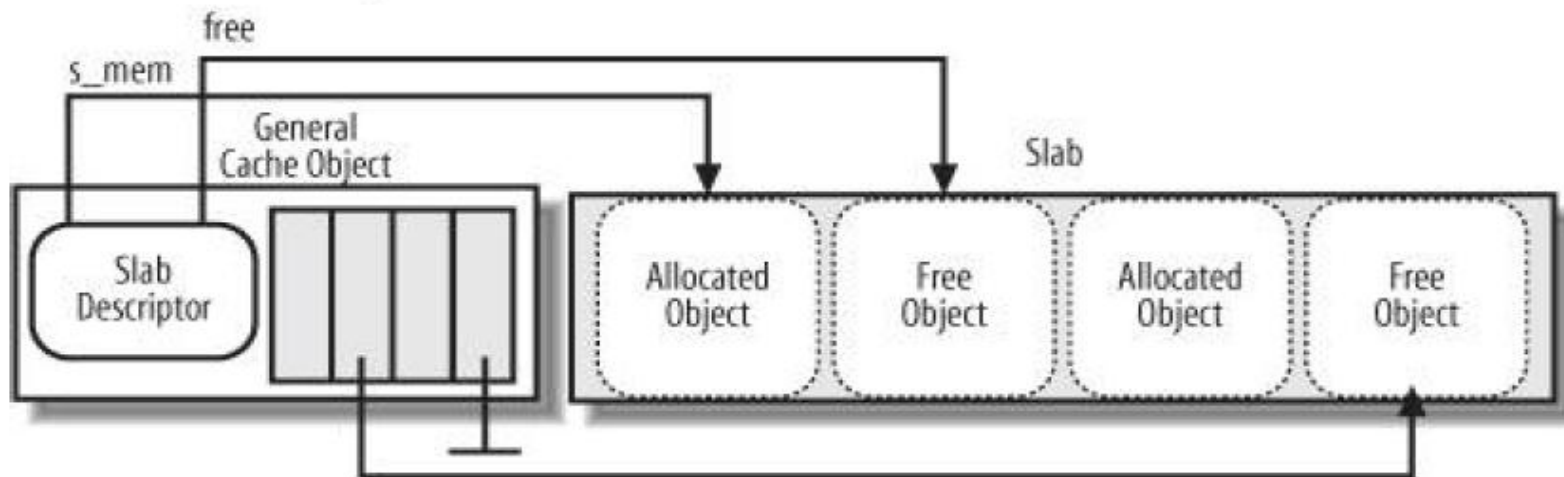
The Slab Allocator

Relationships between slab and object descriptors

Slab with Internal Descriptors



Slab with External Descriptors



The Slab Allocator

- **Aligning Objects in Memory**
- The objects managed by the slab allocator are aligned in memory that is, they are stored in memory cells whose initial physical addresses are multiples of a given constant, which is usually a power of 2. This constant is called the **alignment factor**.
- The largest alignment factor allowed by the slab allocator is 4,096 the page frame size.
- This means that objects can be aligned by referring to either their physical addresses or their linear addresses.
- In both cases, only the 12 least significant bits of the address may be altered by the alignment.
- Usually, microcomputers access memory cells more quickly if their physical addresses are aligned with respect to the word size (that is, to the width of the internal memory bus of the computer).
- Thus, by default, the `kmem_cache_create( )` function aligns objects according to the word size specified by the `BYTES_PER_WORD` macro.
- For 80 x 86 processors, the macro yields the value 4 because the word is 32 bits long.

The Slab Allocator

❑ Slab Coloring

The same hardware cache line maps many different blocks of RAM.

It is seen that objects of the same size end up being stored at the same offset within a cache.

Objects that have the same offset within different slabs will, with a relatively high probability, end up mapped in the same cache line.

The cache hardware might therefore waste memory cycles transferring two objects from the same cache line back and forth to different RAM locations, while other cache lines go underutilized.

The slab allocator tries to reduce this unpleasant cache behavior by a policy called slab coloring : different arbitrary values called colors are assigned to the slabs.

The Slab Allocator

❑ Slab Coloring

- Before examining slab coloring, we have to look at the layout of objects in the cache. Let's consider a cache whose objects are aligned in RAM.
- This means that the object address must be a multiple of a given positive value, say aln .
- Even taking the alignment constraint into account, there are many possible ways to place objects inside the slab.
- The choices depend on decisions made for the following variables:

num	Number of objects that can be stored in a slab (its value is in the num field of the cache descriptor).
osize	Object size, including the alignment bytes.
dsize	Slab descriptor size plus all object descriptors size, rounded up to the smallest multiple of the hardware cache line size. Its value is equal to 0 if the slab and object descriptors are stored outside of the slab.
free	Number of unused bytes (bytes not assigned to any object) inside the slab.

- The total length in bytes of a slab can then be expressed as:
$$\text{slab length} = (\text{num} \times \text{osize}) + \text{dsize} + \text{free}$$
- free is always smaller than osize, because otherwise, it would be possible to place additional objects inside the slab. However, free could be greater than aln .

The Slab Allocator

❑ Slab Coloring

- The slab allocator takes advantage of the free unused bytes to color the slab.
- The term "color" is used simply to subdivide the slabs and allow the memory allocator to spread objects out among different linear addresses.
- In this way, the kernel obtains the best possible performance from the microprocessor's hardware cache.
- Slabs having different colors store the first object of the slab in different memory locations, while satisfying the alignment constraint.
- The number of available colors is free/aln (this value is stored in the colour field of the cache descriptor).
- Thus, the first color is denoted as 0 and the last one is denoted as $(\text{free} / \text{aln}) - 1$.
- (As a particular case, if free is lower than aln, colour is set to 0, nevertheless all slabs use color 0, thus really the number of colors is one.)
- If a slab is colored with color col, the offset of the first object (with respect to the slab initial address) is equal to $\text{col} \times \text{aln} + \text{dsize}$ bytes.

The Slab Allocator

- Slab with color col and alignment aln

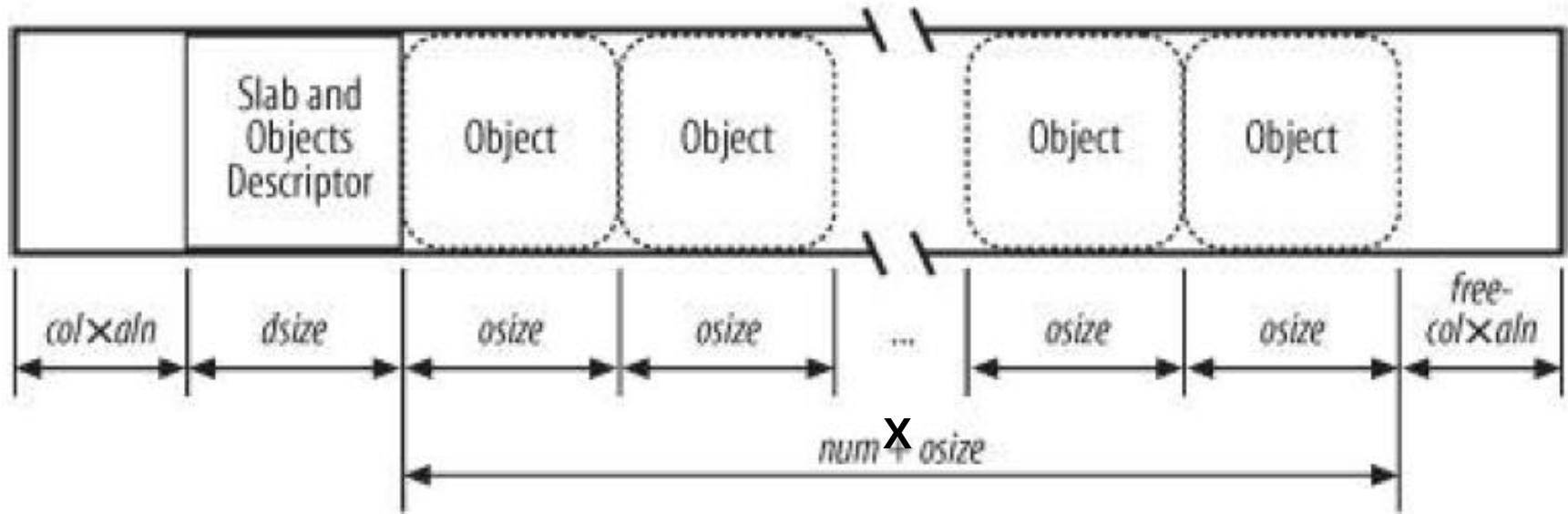


Figure illustrates how the placement of objects inside the slab depends on the slab color. Coloring essentially leads to moving some of the free area of the slab from the end to the beginning.

The Slab Allocator

- Coloring works only when free is large enough.
- Clearly, if no alignment is required for the objects or if the number of unused bytes inside the slab is smaller than the required alignment ($\text{free} < \text{aln}$), the only possible slab coloring is the one that has the color 0 the one that assigns a zero offset to the first object.
- The various colors are distributed equally among slabs of a given object type by storing the current color in a field of the cache descriptor called `colour_next`.
- The `cache_grow( )` function assigns the color specified by `colour_next` to a new slab and then increases the value of this field.
- After reaching `colour`, it wraps around again to 0.
- In this way, each slab is created with a different color from the previous one, up to the maximum available colors.
- The `cache_grow( )` function, moreover, gets the value `aln` from the `colour_off` field of the cache descriptor, computes `dsize` according to the number of objects inside the slab, and finally stores the value `colx aln + dsize` in the `colouroff` field of the slab descriptor.

The Slab Allocator

❑ Allocating a Slab Object

- New objects may be obtained by invoking the **kmem_cache_alloc()** function.
- The parameter **cachep** points to the cache descriptor from which the new free object must be obtained, while the parameter **flag** represents the flags to be passed to the zoned page frame allocator functions, should all slabs of the cache be full.

❑ Freeing a Slab Object

- The **kmem_cache_free()** function releases an object previously allocated by the slab allocator to some kernel function.
- Its parameters are **cachep**, the address of the cache descriptor, and **objp**, the address of the object to be released.

The Slab Allocator

❑ General Purpose Objects

- As stated in "The Buddy System Algorithm," infrequent requests for memory areas are handled through a group of general caches whose objects have geometrically distributed sizes ranging from a minimum of 32 to a maximum of 131,072 bytes.
- Objects of this type are obtained by invoking the **kmalloc()** function.
- Objects obtained by invoking **kmalloc()** can be released by calling **kfree()**:

The Slab Allocator

❑ Memory Pools

- Memory pools are a new feature of Linux
- Basically, a memory pool allows a kernel component such as the block device subsystem to allocate some dynamic memory to be used only in low-on-memory emergencies.
- Memory pools should not be confused with the reserved page frames "The Pool of Reserved Page Frames."
- In fact, those page frames can be used only to satisfy atomic memory allocation requests issued by interrupt handlers or inside critical regions.
- Instead, a memory pool is a reserve of dynamic memory that can be used only by a specific kernel component, namely the "owner" of the pool.

The Slab Allocator

❑ Memory Pools

- The owner does not normally use the reserve; however, if dynamic memory becomes so scarce that all usual memory allocation requests are doomed to fail, the kernel component can invoke, as a last resort, special memory pool functions that dip in the reserve and get the memory needed.
- Thus, creating a memory pool is similar to keeping a reserve of canned foods on hand and using a can opener only when no fresh food is available.
- Often, a memory pool is stacked over the slab allocator that is, it is used to keep a reserve of slab objects.
- Generally speaking, however, a memory pool can be used to allocate every kind of dynamic memory, from whole page frames to small memory areas allocated with `kmalloc()`.
- Therefore, we will generically refer to the memory units handled by a memory pool as "memory elements."

SOC 3010

OPERATING SYSTEM

NON-CONTIGUOUS MEMORY AREA MANAGEMENT

Noncontiguous Memory Area Management

- it is preferable to map memory areas into sets of contiguous page frames, thus making better use of the cache and achieving lower average memory access times.
- Nevertheless, if the requests for memory areas are infrequent, it makes sense to consider an allocation scheme based on noncontiguous page frames accessed through contiguous linear addresses .
- The main advantage of this schema is to avoid external fragmentation, while the disadvantage is that it is necessary to fiddle with the kernel Page Tables.
- Clearly, the size of a noncontiguous memory area must be a multiple of 4,096.
- Linux uses noncontiguous memory areas in several ways for instance, to allocate data structures for active swap areas, to allocate space for a module, or to allocate buffers to some I/O drivers.
- Furthermore, noncontiguous memory areas provide yet another way to make use of high memory page frames

Noncontiguous Memory Area Management

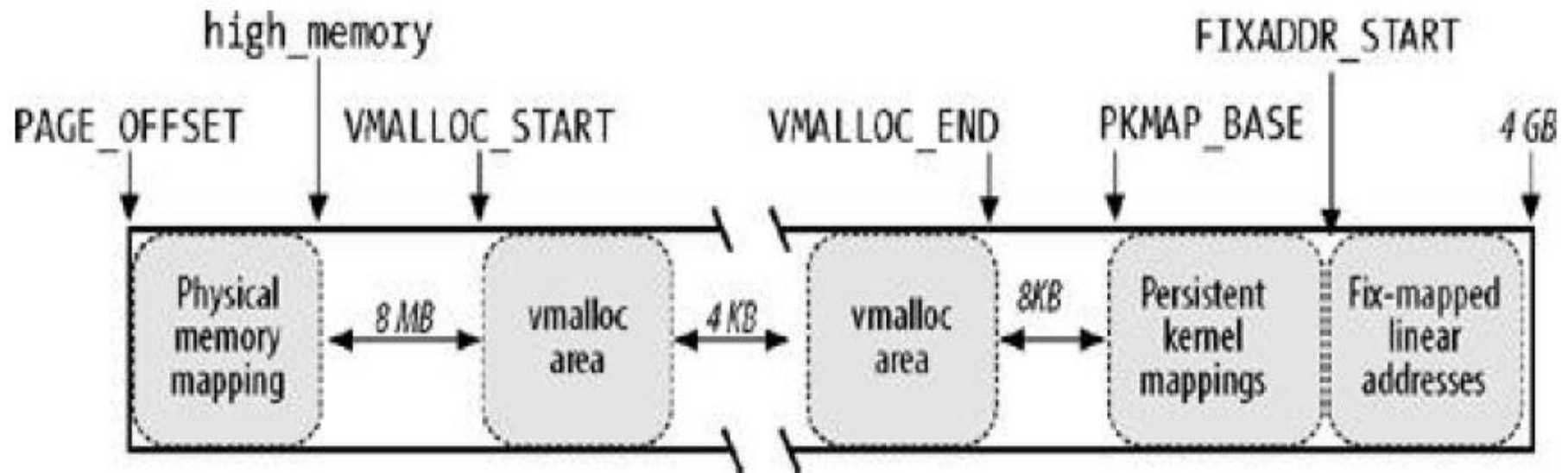
❑ Linear Addresses of Noncontiguous Memory Areas

- To find a free range of linear addresses, we can look in the area starting from `PAGE_OFFSET` (usually `0xc0000000`, the beginning of the fourth gigabyte). Figure shows how the fourth gigabyte linear addresses are used:
- The beginning of the area includes the linear addresses that map the first 896 MB of RAM, the linear address that corresponds to the end of the directly mapped physical memory is stored in the `high_memory` variable.
- The end of the area contains the fix-mapped linear addresses
- Starting from `PKMAP_BASE` we find the linear addresses used for the persistent kernel mapping of high-memory page frames
- The remaining linear addresses can be used for noncontiguous memory areas.
- A safety interval of size 8 MB (macro `VMALLOC_OFFSET`) is inserted between the end of the physical memory mapping and the first memory area; its purpose is to "capture" out-of-bounds memory accesses. For the same reason, additional safety intervals of size 4 KB are inserted to separate noncontiguous memory areas.

Noncontiguous Memory Area Management

- Linear Addresses of Noncontiguous Memory Areas

The linear address interval starting from PAGE_OFFSET



The `VMALLOC_START` macro defines the starting address of the linear space reserved for noncontiguous memory areas, while `VMALLOC_END` defines its ending address.

Noncontiguous Memory Area Management

❑ Allocating a Noncontiguous Memory Area

- The **`vmalloc( )`** function allocates a noncontiguous memory area to the kernel.
- The parameter `size` denotes the size of the requested area.
- If the function is able to satisfy the request, it then returns the initial linear address of the new area; otherwise, it returns a NULL pointer

❑ Releasing a Noncontiguous Memory Area

- The **`vfree( )`** function releases noncontiguous memory areas created by `vmalloc( )` or `vmalloc_32( )`, while the `munmap( )` function releases memory areas created by `mmap( )`.
- Both functions have one parameter the address of the initial linear address of the area to be released; they both rely on the `__munmap( )` function to do the real work.
- The `__munmap( )` function receives two parameters: the address `addr` of the initial linear address of the area to be released, and the flag `deallocate_pages`, which is set if the page frames mapped in the area should be released to the zoned page frame allocator and cleared otherwise,

MEMORY MANAGEMENT

❑ Questions

- * Describe the functions of a zone allocator
- * Explain what is meant by internal Fragmentation. Describe the approach used by Linux Kernel to reduce Internal fragmentation
- * Running a memory area allocation algorithm on top of the buddy algorithm is not particularly efficient. Discuss the better approach used in linux kernel to achieve the efficiency.
- * Explain the following :
 - a) Slab
 - b) object
- * With a suitable schematic, describe the functions of a Slab Allocator. Clearly explain the functions of each component in the slab allocator.
- * What are General and Specific caches?
- * Write the functions for allocating a slab to a cache and destroying a slab from a cache.
- * With the help of a schematic, explain the relationship between slab and object descriptors.
- * Explain what is meant by aligning objects in memory. Why is it required?
- * Explain the need for Slab Coloring mechanism.
- * Describe the approach used in slab coloring mechanism to reduce higher hardware cache misses.
- * Write the expression for computing the length of a slab clearly explaining the various parameters used in the expression.
- * With the help of a schematic, illustrate how the placement of objects inside the slab depends on the slab color.
- * Explain what is meant by General Purpose object.
- * Describe the memory Pool of a kernel component. Indicate clearly where it is located.
- * Explain the need for Non-Contiguous Memory Area.
- * Describe Non-Contiguous Memory Area Management in Linux
- *

MEMORY MANAGEMENT



□ Questions

- * Draw the memory layout of the kernel memory (fourth GB of RAM) showing all the memory areas (NORMAL Zone (Physical Memory Mapping), Non-Contiguous Memory Area, Memory areas used for Persistent Kernel mappings and fix-mapped linear addresses), their sizes, starting and ending addresses of each of these memory areas. Also show how non-contiguous blocks are allocated in the memory area.
- * Provide the addresses stored in following parameters :
 - a) PAGE_OFFSET b) PKMAP_BASE
- * State the need for inserting safety interval between the end of the physical memory mapping and the first memory area and additional safety intervals to separate non-contiguous memory areas in the kernel memory. Also indicate the size of these safety intervals.
- * What are the functions used to allocate and release Non-contiguous memory Areas.
- * Given the following Memory layout containing 10 memory areas with their sizes, determine the start and end address of each of these memory areas assuming that the start address of first memory area (MA1) is 0x70000000.

MA1	MA2	MA3	MA4	MA5	MA6	MA7	MA8	MA9	MA10
364MB	184MB	296MB	88MB	540MB	56MB	32MB	128MB	256MB	52MB

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 8 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

SOC 3010
OPERATING SYSTEM

END OF
WEEK 14 LECTURE 1

MEMORY
MANAGEMENT

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

